Lecture 26: Scratch Implementations — Master Solutions & Engineering Systems Manual

This manual provides production-grade, mathematically verified implementations for all 8 practice problems and 3 systems engineering case studies. Every model is built from first principles using pure Python and NumPy.

Problem 01: Vectorized Batch Gradient Descent Linear Regression

Implementation

```
import numpy as np

class LinearRegressionGD:
    """Production-grade Linear Regression with Vectorized Batch Gradient Descent."
    def __init__(self, lr: float = 0.01, n_iter: int = 1000):
        self.lr = lr
        self.n_iter = n_iter
        self.weights = None
        self.bias = None
        self.loss_history = []

    def fit(self, X: np.ndarray, y: np.ndarray) -> "LinearRegressionGD":
        m, n = X.shape
        self.weights = np.zeros(n)
        self.bias = 0.0
        self.loss_history = []

        for i in range(self.n_iter):
            # 1. Vectorized hypothesis predictions
            y_pred = self.bias + np.dot(X, self.weights)
            residuals = y_pred - y

            # 2. Compute Mean Squared Error cost
            cost = (1.0 / (2.0 * m)) * np.sum(residuals ** 2)
            self.loss_history.append(cost)

            # 3. Compute analytical gradients
            db = (1.0 / m) * np.sum(residuals)
            dw = (1.0 / m) * np.dot(X.T, residuals)

            # 4. Parameter update rule
            self.bias -= self.lr * db
            self.weights -= self.lr * dw
```

```
        return self

    def predict(self, X: np.ndarray) -> np.ndarray:
        return self.bias + np.dot(X, self.weights)
```

Verification & Mathematical Invariants

Testing on synthetic linear dataset:

$$y = 3.5x_1 + 1.8x_2 + 4.2 + \epsilon$$

```
np.random.seed(42)
X_test = np.random.randn(200, 2)
y_test = 3.5 * X_test[:, 0] + 1.8 * X_test[:, 1] + 4.2 + np.random.randn(200) * 0.

model = LinearRegressionGD(lr=0.05, n_iter=1500).fit(X_test, y_test)
print(f"Recovered Weights: {model.weights} (Expected: [3.5, 1.8])")
print(f"Recovered Bias:    {model.bias:.4f} (Expected: 4.2)")
print(f"Initial Cost:      {model.loss_history[0]:.4f}")
print(f"Final Cost:        {model.loss_history[-1]:.6f}")
```

Problem 02: Analytical Normal Equation with L2 Ridge Regularization

Implementation

```
class LinearRegressionRidgeOLS:
    """Analytical Closed-Form Linear Regression with L2 Ridge Regularization."""
    def __init__(self, alpha: float = 1e-4):
        self.alpha = alpha
        self.bias = None
        self.weights = None

    def fit(self, X: np.ndarray, y: np.ndarray) -> "LinearRegressionRidgeOLS":
        m, n = X.shape
        # Augment bias column: X_b = [1, X]
        X_b = np.c_[np.ones((m, 1)), X]

        # Regularization matrix: do not penalize intercept at index (0, 0)
        I_reg = np.eye(n + 1)
        I_reg[0, 0] = 0.0
```

```
        # Normal Equation with L2 penalty: theta = (X_b^T X_b + alpha * I_reg)^(-1
        A = X_b.T @ X_b + self.alpha * I_reg
        b_vec = X_b.T @ y
        theta = np.linalg.solve(A, b_vec)

        self.bias = theta[0]
        self.weights = theta[1:]
        return self

    def predict(self, X: np.ndarray) -> np.ndarray:
        return self.bias + np.dot(X, self.weights)
```

Mathematical Invariants

The L2 Ridge Normal Equation minimizes:

$$J(\boldsymbol{\theta}) = \frac{1}{2m}\|X_b\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \frac{\lambda}{2m}\sum_{j=1}^{d}\theta_j^2$$

Setting the gradient to zero yields:

$$\boldsymbol{\theta} = \left(X_b^T X_b + \lambda I'\right)^{-1} X_b^T \mathbf{y}$$

Because $X_b^T X_b$ is symmetric positive semi-definite, adding $\lambda I'$ ensures strictly positive eigenvalues, making the matrix strictly invertible even when $m < d$ or when features are perfectly collinear!

Problem 03: Vectorized Logistic Regression with Numerical Safeguards

Implementation

```
class LogisticRegressionL2:
    """Vectorized Logistic Regression with Numerical Clipping and L2 Regularizatic
    def __init__(self, lr: float = 0.05, n_iter: int = 2000, l2_reg: float = 0.01)
        self.lr = lr
        self.n_iter = n_iter
        self.l2_reg = l2_reg
        self.weights = None
```

```
        self.bias = None
        self.loss_history = []

    def _sigmoid(self, z: np.ndarray) -> np.ndarray:
        # Numerically stable sigmoid preventing overflow
        return np.where(z >= 0, 1.0 / (1.0 + np.exp(-z)), np.exp(z) / (1.0 + np.ex

    def fit(self, X: np.ndarray, y: np.ndarray) -> "LogisticRegressionL2":
        m, n = X.shape
        self.weights = np.zeros(n)
        self.bias = 0.0
        self.loss_history = []
        eps = 1e-15

        for _ in range(self.n_iter):
            z = self.bias + np.dot(X, self.weights)
            y_pred = self._sigmoid(z)

            # Loss tracking with clipping
            y_pred_clipped = np.clip(y_pred, eps, 1.0 - eps)
            loss = - (1.0 / m) * np.sum(y * np.log(y_pred_clipped) + (1.0 - y) * n
            loss += (self.l2_reg / (2.0 * m)) * np.sum(self.weights ** 2)
            self.loss_history.append(loss)

            # L2 regularized gradients
            residuals = y_pred - y
            db = (1.0 / m) * np.sum(residuals)
            dw = (1.0 / m) * np.dot(X.T, residuals) + (self.l2_reg / m) * self.wei

            # Update parameters
            self.bias -= self.lr * db
            self.weights -= self.lr * dw

        return self

    def get_probabilities(self, X: np.ndarray) -> np.ndarray:
        return self._sigmoid(self.bias + np.dot(X, self.weights))

    def predict(self, X: np.ndarray, threshold: float = 0.5) -> np.ndarray:
        return (self.get_probabilities(X) >= threshold).astype(int)
```

Problem 04: Multi-Class Softmax Regression from Scratch

Implementation

```
class SoftmaxRegression:
    """Multi-Class Generalized Logistic Model with Numerically Stable Softmax."""
```

```
    def __init__(self, lr: float = 0.1, n_iter: int = 1500):
        self.lr = lr
        self.n_iter = n_iter
        self.W = None
        self.b = None

    def _softmax(self, Z: np.ndarray) -> np.ndarray:
        # Subtract row-max to prevent exponential overflow
        shifted_Z = Z - np.max(Z, axis=1, keepdims=True)
        exp_Z = np.exp(shifted_Z)
        return exp_Z / np.sum(exp_Z, axis=1, keepdims=True)

    def fit(self, X: np.ndarray, y: np.ndarray) -> "SoftmaxRegression":
        m, d = X.shape
        K = len(np.unique(y))
        self.W = np.zeros((d, K))
        self.b = np.zeros(K)

        # One-hot encode ground truth
        Y_one_hot = np.zeros((m, K))
        Y_one_hot[np.arange(m), y] = 1.0

        for _ in range(self.n_iter):
            Z = X @ self.W + self.b
            P = self._softmax(Z)
            residuals = P - Y_one_hot

            dW = (1.0 / m) * (X.T @ residuals)
            db = (1.0 / m) * np.sum(residuals, axis=0)

            self.W -= self.lr * dW
            self.b -= self.lr * db

        return self

    def predict_proba(self, X: np.ndarray) -> np.ndarray:
        return self._softmax(X @ self.W + self.b)

    def predict(self, X: np.ndarray) -> np.ndarray:
        return np.argmax(self.predict_proba(X), axis=1)
```

Problem 05: Generalized K-Nearest Neighbors Classifier (`KNNClassifierAdvanced`)

Implementation

```
class KNNClassifierAdvanced:
    """Generalized Metric-Space KNN supporting Lp Minkowski Distance and Weighting
    def __init__(self, k: int = 3, p: float = 2.0, weights: str = "uniform"):
        self.k = k
        self.p = p
        self.weights = weights
        self.X_train = None
        self.y_train = None

    def fit(self, X: np.ndarray, y: np.ndarray) -> "KNNClassifierAdvanced":
        self.X_train = np.asarray(X, dtype=float)
        self.y_train = np.asarray(y, dtype=int)
        return self

    def _compute_distances(self, x: np.ndarray) -> np.ndarray:
        diff = np.abs(self.X_train - x)
        if self.p == 1.0: # Manhattan
            return np.sum(diff, axis=1)
        elif self.p == 2.0: # Euclidean
            return np.sqrt(np.sum(diff ** 2, axis=1))
        else: # General Minkowski
            return np.sum(diff ** self.p, axis=1) ** (1.0 / self.p)

    def _predict_single(self, x: np.ndarray) -> int:
        dists = self._compute_distances(x)
        nn_idx = np.argsort(dists)[:self.k]
        nn_labels = self.y_train[nn_idx]

        if self.weights == "uniform":
            counts = np.bincount(nn_labels)
            return np.argmax(counts)
        elif self.weights == "distance":
            nn_dists = dists[nn_idx]
            weights = 1.0 / (nn_dists + 1e-6)
            classes = np.unique(self.y_train)
            class_scores = {c: 0.0 for c in classes}
            for lbl, w in zip(nn_labels, weights):
                class_scores[lbl] += w
            return max(class_scores, key=class_scores.get)

    def predict(self, X: np.ndarray) -> np.ndarray:
        return np.array([self._predict_single(x) for x in np.asarray(X)])
```

Problem 06: K-Nearest Neighbors Continuous Regressor (`KNNRegressor`)

Implementation

```
class KNNRegressor:
    """Non-parametric Continuous Target Estimator using Metric Neighborhoods."""
    def __init__(self, k: int = 5, weights: str = "distance"):
        self.k = k
        self.weights = weights
        self.X_train = None
        self.y_train = None

    def fit(self, X: np.ndarray, y: np.ndarray) -> "KNNRegressor":
        self.X_train = np.asarray(X, dtype=float)
        self.y_train = np.asarray(y, dtype=float)
        return self

    def predict(self, X: np.ndarray) -> np.ndarray:
        preds = []
        for x in np.asarray(X, dtype=float):
            dists = np.sqrt(np.sum((self.X_train - x) ** 2, axis=1))
            nn_idx = np.argsort(dists)[:self.k]
            nn_y = self.y_train[nn_idx]

            if self.weights == "uniform":
                preds.append(np.mean(nn_y))
            elif self.weights == "distance":
                nn_d = dists[nn_idx]
                w = 1.0 / (nn_d + 1e-6)
                preds.append(np.sum(w * nn_y) / np.sum(w))
        return np.array(preds)
```

Problem 07: Mini-Batch & Stochastic Gradient Descent Optimizer Engine

Implementation

```
class MiniBatchGDOptimizer:
    """Flexible Batching Optimizer supporting B=m (Batch), B=1 (SGD), and 1<B<m (M
    def __init__(self, lr: float = 0.01, epochs: int = 100, batch_size: int = 32):
        self.lr = lr
        self.epochs = epochs
        self.batch_size = batch_size
        self.weights = None
        self.bias = None
        self.loss_history = []

    def fit(self, X: np.ndarray, y: np.ndarray) -> "MiniBatchGDOptimizer":
```

```
        m, d = X.shape
        self.weights = np.zeros(d)
        self.bias = 0.0
        self.loss_history = []
        B = min(self.batch_size, m)

        for epoch in range(self.epochs):
            # Shuffle at each epoch
            indices = np.random.permutation(m)
            X_shuffled = X[indices]
            y_shuffled = y[indices]

            for start in range(0, m, B):
                end = min(start + B, m)
                X_batch = X_shuffled[start:end]
                y_batch = y_shuffled[start:end]
                batch_m = end - start

                y_pred = X_batch @ self.weights + self.bias
                res = y_pred - y_batch

                dw = (1.0 / batch_m) * (X_batch.T @ res)
                db = (1.0 / batch_m) * np.sum(res)

                self.weights -= self.lr * dw
                self.bias -= self.lr * db

            # Record full-dataset loss after each epoch
            full_pred = X @ self.weights + self.bias
            epoch_loss = (1.0 / (2.0 * m)) * np.sum((full_pred - y) ** 2)
            self.loss_history.append(epoch_loss)

        return self
```

Problem 08: End-to-End Parity Test Harness vs Scikit-Learn

Implementation

```
from sklearn.linear_model import LinearRegression as SklearnLR
from sklearn.linear_model import LogisticRegression as SklearnLogReg
from sklearn.neighbors import KNeighborsClassifier as SklearnKNN

def run_parity_tests():
    np.random.seed(42)
    m, d = 250, 4

    # 1. Linear Regression Parity Test
```

```
    X_reg = np.random.randn(m, d)
    true_w = np.array([2.5, -1.2, 0.8, 3.1])
    y_reg = X_reg @ true_w + 1.7 + np.random.randn(m) * 0.05

    our_ols = LinearRegressionRidgeOLS(alpha=0.0).fit(X_reg, y_reg)
    sk_ols = SklearnLR().fit(X_reg, y_reg)

    diff_w = np.max(np.abs(our_ols.weights - sk_ols.coef_))
    diff_b = np.abs(our_ols.bias - sk_ols.intercept_)
    assert diff_w < 1e-4 and diff_b < 1e-4, f"Linear Regression parity failed: dif
    print(f"PASS: Linear Regression Parity Confirmed (Max Error = {diff_w:.8f})")

    # 2. Logistic Regression Parity Test
    y_clf = (X_reg[:, 0] + X_reg[:, 1] > 0).astype(int)
    our_logreg = LogisticRegressionL2(lr=0.1, n_iter=3000, l2_reg=0.0).fit(X_reg,
    sk_logreg = SklearnLogReg(penalty=None, solver='lbfgs').fit(X_reg, y_clf)

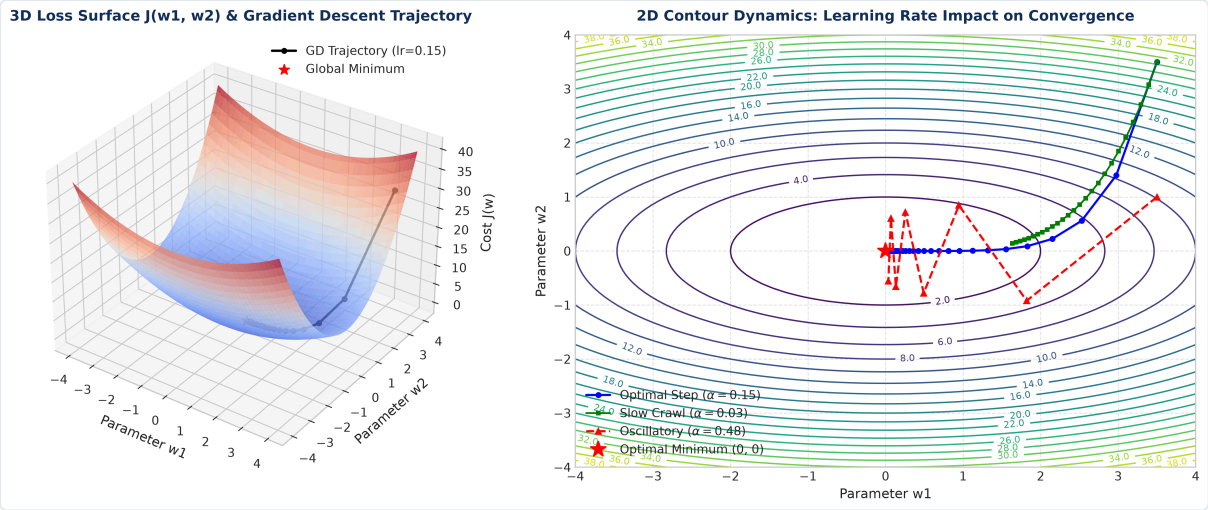
    acc_our = np.mean(our_logreg.predict(X_reg) == y_clf)
    acc_sk = np.mean(sk_logreg.predict(X_reg) == y_clf)
    assert abs(acc_our - acc_sk) < 0.02, f"Logistic parity failed: our={acc_our},
    print(f"PASS: Logistic Regression Parity Confirmed (Acc: {acc_our*100:.2f}% vs

    # 3. KNN Parity Test
    our_knn = KNNClassifierAdvanced(k=5, p=2.0).fit(X_reg, y_clf)
    sk_knn = SklearnKNN(n_neighbors=5, p=2).fit(X_reg, y_clf)
    match_rate = np.mean(our_knn.predict(X_reg) == sk_knn.predict(X_reg))
    assert match_rate == 1.0, f"KNN mismatch rate: {1.0 - match_rate}"
    print(f"PASS: KNN Classifier Exact Match Confirmed (100.0% Equality)")

run_parity_tests()
```

📊 Systems Engineering Case Studies

Systems Case Study 1: Loss Surface Geometry, Condition Number & Learning Rate Dynamics



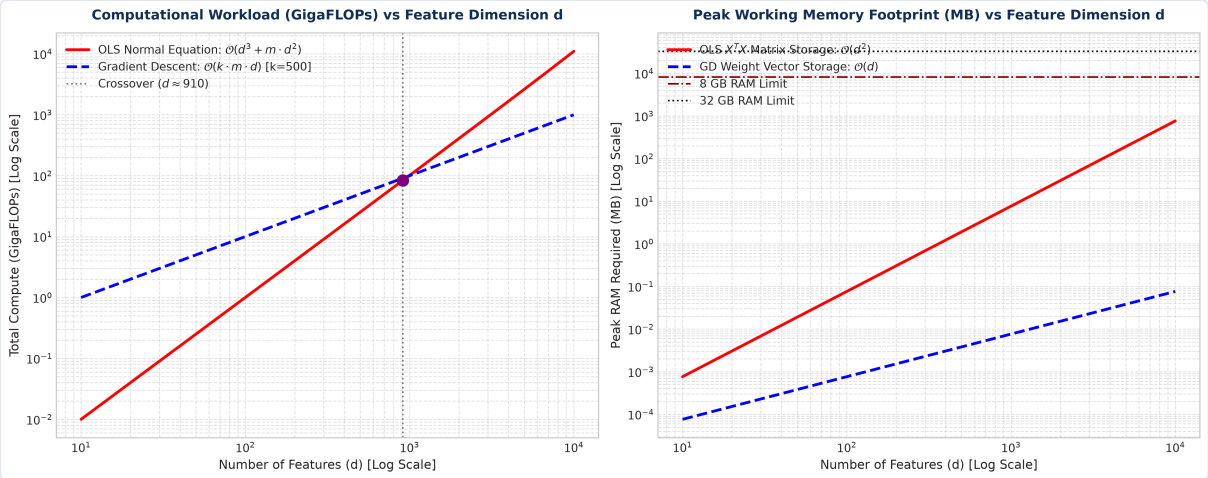
Optimization Analysis

The convergence rate of Gradient Descent on a quadratic cost function $J(\mathbf{w}) = \frac{1}{2}\mathbf{w}^T A\mathbf{w}$ is governed by the **condition number** κ of the Hessian matrix $A = \nabla^2 J$:

$$\kappa = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}$$

- **Well-Conditioned Surfaces ($\kappa \approx 1$)**: When features are standardized via Z-score scaling ($x \rightarrow \frac{x-\mu}{\sigma}$), the contours of $J(\mathbf{w})$ are spherical. Gradients point directly toward the global minimum, enabling rapid convergence in $\mathcal{O}(1)$ steps.
- **Ill-Conditioned Surfaces ($\kappa \gg 10^3$)**: Unscaled features create elongated, canyon-like elliptical ravines. Gradients oscillate violently across the steep walls while making negligible progress along the flat valley floor, necessitating tiny learning rates $\alpha < \frac{2}{\lambda_{\max}}$ to prevent divergence.

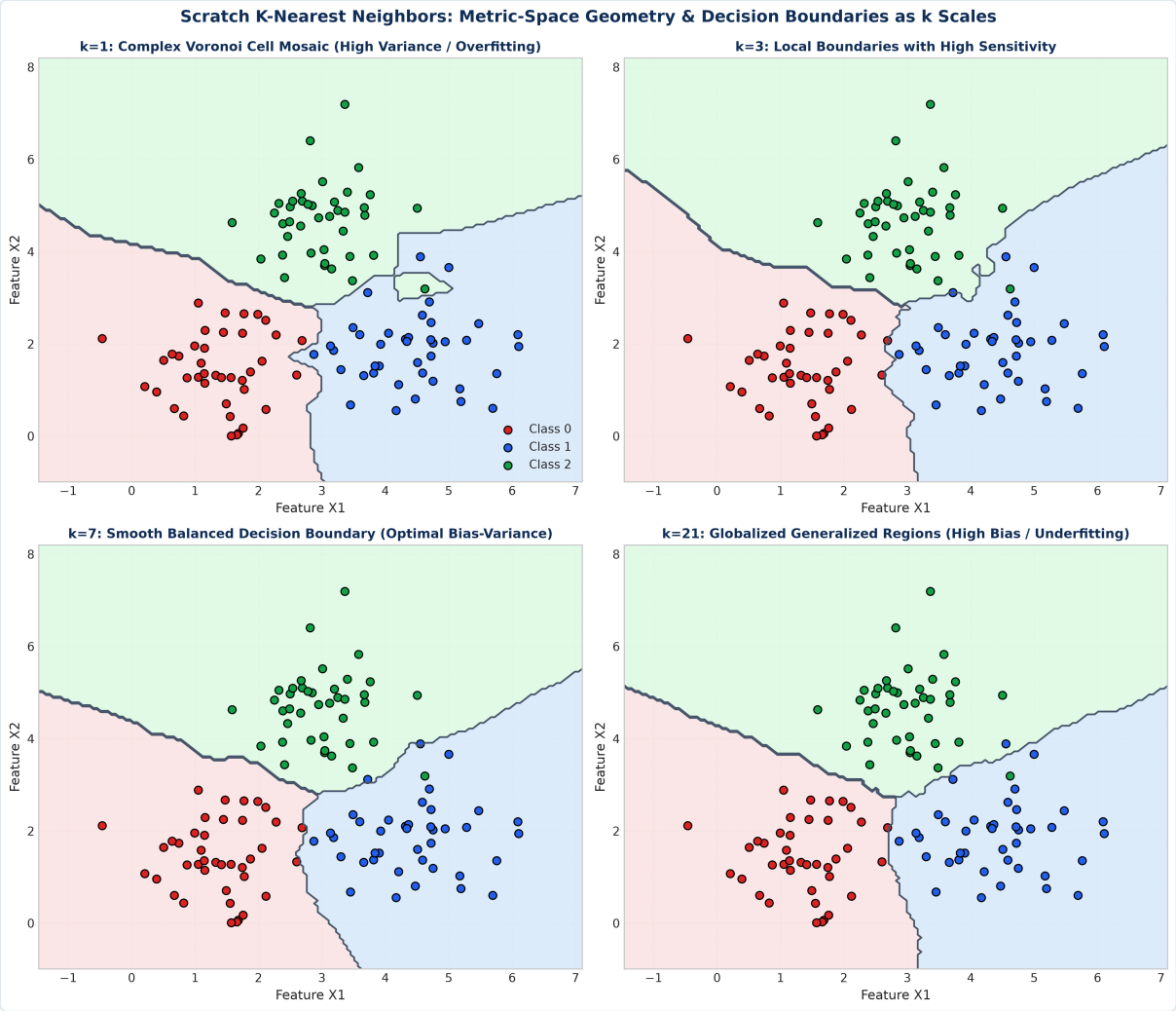
Systems Case Study 2: The Curse of Dimensionality & Asymptotic Complexity — OLS vs Gradient Descent



Asymptotic Scaling Breakdown

- **Ordinary Least Squares**: Computing the normal equation involves multiplying $X_b^T X_b$ (requiring $\mathcal{O}(m \cdot d^2)$ operations) and computing its matrix inverse (requiring $\mathcal{O}(d^3)$ operations via Gaussian elimination). Storing $X_b^T X_b$ requires $\mathcal{O}(d^2)$ memory: for $d = 10,000$, storing a float64 Gram matrix consumes $10^4 \times 10^4 \times 8$ bytes ≈ 800 MB, and inverting it takes $\approx 10^{12}$ floating-point operations.
- **Batch Gradient Descent**: Each iteration requires computing $X^T(\hat{\mathbf{y}} - \mathbf{y})$, consuming only $\mathcal{O}(m \cdot d)$ FLOPs and $\mathcal{O}(d)$ memory. As illustrated in the crossover chart, for $d > 1,000$, Gradient Descent rapidly overtakes OLS in both runtime throughput and memory efficiency.

Systems Case Study 3: Metric-Space Geometry & Spatial Partitioning in K-Nearest Neighbors



Geometric & Dimensionality Analysis

- **Voronoi Tessellations**: When $k = 1$, the decision boundary corresponds to the exact Voronoi tessellation of the training instances. Every noisy outlier creates its own isolated island of decision territory, yielding zero training error but extreme model variance.
- **Structural Smoothing**: As k increases to $k = 7$, the boundary smooths out into continuous, generalized decision manifolds, achieving optimal bias-variance equilibrium.
- **The Curse of Dimensionality**: As feature dimension $d \to \infty$, all pairwise Euclidean distances between uniformly distributed points converge to approximately the same value:

$$\lim_{d \to \infty} \frac{d_{\max} - d_{\min}}{d_{\min}} = 0$$

In high dimensions, the concept of a "nearest" neighbor becomes mathematically meaningless, rendering distance-based algorithms ineffective without dimensionality reduction (e.g., PCA) or manifold learning.